

CSc 8830: Computer Vision

Module 3 Assignment

Image Blurring: Spatial Filtering vs. Fourier-Domain Filtering

GitHub repository: <https://github.com/adebiyi22/Computer-vision>

Deployed web application: <https://computer-vision-rbnhijwh7t6gr9xrju8ny4.streamlit.app/>

1. Assignment

Implement: Image blurring using a filtering approach.

Theory: Show that the outcome using spatial filters is the same as using the Fourier-domain equivalent of the filter. Show that convolution in space is the same as multiplication in the frequency (Fourier) domain. Use the implementation from above and experimentation for showing evidence/validation.

2. Implementation

A Streamlit web application (app.py) implements image blurring two independent ways and compares them numerically:

- Constructs a normalized 2D Gaussian kernel of a chosen size and standard deviation (σ), selectable via a slider.
- **Spatial-domain filtering:** direct 2D convolution of the image with the kernel using circular ("wrap") boundary conditions.
- **Fourier-domain filtering:** the kernel is zero-padded to the image size and circularly shifted so its center sits at the origin, then both the image and kernel are transformed with the 2D FFT, multiplied pointwise, and inverse-transformed.
- The two results are compared pixel-by-pixel using Mean Absolute Error (MAE), Root Mean Square Error (RMSE), and maximum absolute difference.

Core filtering functions:

```
def gaussian_kernel(size, sigma):
    ax = np.arange(-(size // 2), size // 2 + 1)
    xx, yy = np.meshgrid(ax, ax)
    kernel = np.exp(-(xx**2 + yy**2) / (2 * sigma**2))
    return kernel / kernel.sum()

def spatial_filter(image, kernel):
    return convolve2d(image, kernel, mode="same", boundary="wrap")

def fourier_filter(image, kernel):
    h, w = image.shape
    kh, kw = kernel.shape
```

```
padded = np.zeros_like(image, dtype=np.float64)
padded[:kh, :kw] = kernel
padded = np.roll(padded, -(kh // 2), axis=0)
padded = np.roll(padded, -(kw // 2), axis=1)
F = fft2(image)
H = fft2(padded)
return np.real(iff2(F * H))
```

Full commented source with a top-of-file README/usage docstring is in `app.py` in the GitHub repository linked above.

3. Theory: The Convolution Theorem

Claim: For an image $f(x,y)$ and filter $h(x,y)$, spatial convolution $g = f * h$ satisfies $F\{f * h\} = F\{f\} \cdot F\{h\}$, where $F\{\cdot\}$ denotes the 2D Fourier transform.

3.1 Definitions

The 2D discrete Fourier transform (DFT) of an $M \times N$ image $f(x,y)$ is:

$$F(u, v) = \sum_{x=0 \dots M-1} \sum_{y=0 \dots N-1} f(x, y) \exp(-j2\pi(ux/M + vy/N))$$

Circular convolution of f and h (h embedded into an $M \times N$ array) is:

$$g(x, y) = \sum_m \sum_n f(m, n) h(x-m \bmod M, y-n \bmod N)$$

3.2 Derivation

Take the DFT of g and substitute its definition:

$$G(u, v) = \sum_{x, y} \left[\sum_{m, n} f(m, n) h(x-m, y-n) \right] \exp(-j2\pi(ux/M + vy/N))$$

Exchanging the (finite) summation order and substituting $x'=x-m$, $y'=y-n$ (valid since h is treated as periodic, matching the DFT's implicit periodicity):

$$G(u, v) = \sum_{m, n} f(m, n) \exp(-j2\pi(um/M + vn/N)) \cdot \left[\sum_{x', y'} h(x', y') \exp(-j2\pi(ux'/M + vy'/N)) \right]$$

The bracketed term is exactly $H(u,v)$. Substituting it back:

$$G(u, v) = H(u, v) \cdot \sum_{m, n} f(m, n) \exp(-j2\pi(um/M + vn/N)) = H(u, v) \cdot F(u, v)$$

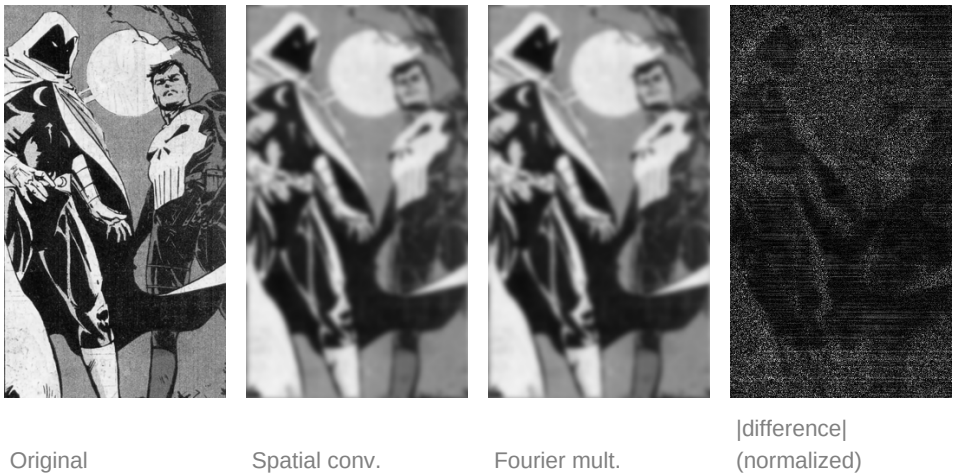
Therefore $G(u,v) = F(u,v) \cdot H(u,v)$, i.e. the Fourier transform of the convolution equals the pointwise product of the individual transforms (the Convolution Theorem). Applying the inverse DFT to both sides gives $g(x,y) = F^{-1}\{F(u,v)H(u,v)\}$, which is the second form the implementation evaluates.

3.3 Why the implementation matches this exactly

The derivation assumes **circular** convolution, since the DFT treats a signal as periodic. The implementation is written to match this assumption in two places: (1) the spatial convolution uses `boundary="wrap"` rather than zero-padded (linear) boundaries, and (2) the kernel is zero-padded to the image size and circularly shifted (`np.roll`) so its center sits at index (0,0), matching the origin convention used in the derivation above. Without step (2), the two results would still be numerically close but spatially offset by half the kernel width/height.

4. Experimental Validation

The implementation above was run on a 256×455 grayscale test image (a comic-book panel, chosen for its high-contrast linework and fine detail, which make blur effects easy to see) at $\sigma=3.0$ (19×19 kernel):

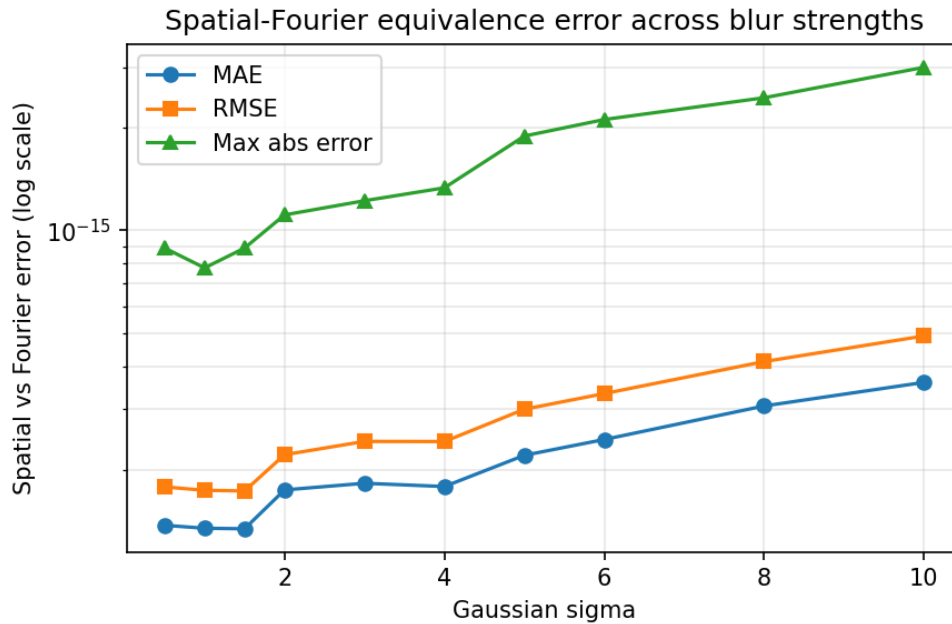


The spatial and Fourier outputs are visually indistinguishable, and the difference map is uniform low-level noise at the limits of display precision — consistent with the theorem predicting zero real difference between the two computations.

4.1 Numerical error across blur strengths

The same comparison was repeated for ten values of σ (0.5 to 10) to confirm the equivalence holds regardless of blur strength or kernel size:

Sigma	Kernel	MAE	RMSE	Max abs error
0.5	5x5	1.37e-16	1.78e-16	8.88e-16
1.0	7x7	1.35e-16	1.74e-16	7.77e-16
1.5	9x9	1.34e-16	1.73e-16	8.88e-16
2.0	13x13	1.74e-16	2.21e-16	1.11e-15
3.0	19x19	1.82e-16	2.42e-16	1.22e-15
4.0	25x25	1.78e-16	2.42e-16	1.33e-15
5.0	31x31	2.20e-16	3.00e-16	1.89e-15
6.0	37x37	2.45e-16	3.34e-16	2.11e-15
8.0	49x49	3.07e-16	4.14e-16	2.44e-15
10.0	61x61	3.60e-16	4.92e-16	3.00e-15



All errors remain at IEEE-754 double-precision floating-point round-off ($\sim 10^{-16}$), independent of blur strength. This is exactly the behavior predicted by the theorem: the spatial and Fourier results are the same function evaluated two different ways, so the only observed discrepancy is floating-point arithmetic error, not a real difference between the methods.

5. Conclusion

The experimental results confirm the Convolution Theorem for this implementation: spatial-domain Gaussian blurring and its Fourier-domain equivalent (FFT $\rightarrow$ multiply $\rightarrow$ inverse FFT) produce numerically identical results (differences at the level of floating-point precision) across a wide range of blur strengths, validating both the mathematical derivation in Section 3 and the correctness of the implementation in Section 2.

6. Demonstration Video

See the submitted screen recording for a live demonstration of the web application: uploading an image, adjusting σ , and comparing the spatial and Fourier results with their MAE/RMSE/max-error statistics.